



Projet de formation - Option B

Monde de Dév - MVP d'un réseau social pour développeurs

Annexe : Rapport de revue technique

Auteur : Emmanuel Hesry

Version : 1.0.0

Date : 31/08/2026

Statut : version finale

Sommaire

Annexe : Rapport de revue technique.....	1
Sommaire.....	1
1. Synthèse.....	2
1.1 Forces constatées.....	2
1.2 Axes d'amélioration.....	2
1.3 Recommandation prioritaire.....	2
2. Périmètre et méthode de revue.....	3
3. Architecture et conception.....	3
3.1 Backend.....	3
3.2 Performance des accès aux données.....	3
3.3 Frontend.....	4
4. Sécurité et protection des données.....	4
5. Qualité du code et corrections réalisées.....	4
6. Tests et automatisation.....	5
7. Axes d'amélioration et recommandations.....	5
7.1 Évolutions fonctionnelles.....	5
7.2 Améliorations techniques et d'exploitation.....	6
8. Conclusion.....	6

1. Synthèse

La solution présente une architecture cohérente avec le périmètre du MVP. Les domaines métier sont séparés côté backend, le frontend est organisé par fonctionnalités et l'API constitue une frontière claire entre les deux couches. Les mécanismes d'authentification, de validation, de gestion des erreurs et de tests sont en place.

Les principaux points forts sont la sécurité de session fondée sur un cookie JWT HttpOnly avec protection CSRF, les tests à plusieurs niveaux et les contrôles locaux de qualité. Les améliorations prioritaires pour une suite du produit concernent l'automatisation CI, la formalisation des règles de rétention des données et l'évolution fonctionnelle au-delà du MVP.

1.1 Forces constatées

- Architecture front et back modulaire, organisée par fonctionnalité ou domaine.
- Contrats REST documentés dans Swagger, DTO distincts des entités et erreurs centralisées.
- Authentification par cookie HttpOnly, CSRF, validations et contrôles d'accès.
- Tests unitaires, d'intégration, end-to-end et API, complétés par formatage, couverture et SonarQube Cloud.
- Requêtes du fil et des commentaires conçues pour éviter le chargement N+1 des auteurs et thèmes.

1.2 Axes d'amélioration

- Absence d'intégration continue pour rejouer automatiquement les contrôles.
- Absence de pagination et de mesure de charge sur le fil.
- Absence de politique formalisée de suppression, d'export et de conservation des données personnelles.
- Absence de supervision d'exploitation et d'alertes de production.

1.3 Recommandation prioritaire

Avant toute évolution fonctionnelle, mettre en place une intégration continue qui exécute les tests, les contrôles de formatage et la génération des rapports sur chaque changement. Elle rendrait la qualité actuelle reproductible sans contrôle manuel systématique.

2. Périmètre et méthode de revue

- Organisation des modules Angular et Spring Boot.
- Échanges REST, sécurité et données.
- Lisibilité, conventions et documentation du code.
- Tests unitaires, d'intégration, end-to-end et API.
- Corrections visibles dans l'historique Git.

Les conclusions s'appuient sur le code versionné, les README, les migrations Flyway, la documentation OpenAPI, les rapports de couverture générés et les commandes de vérification. Les taux de couverture et résultats détaillés sont reportés séparément dans le rapport de tests afin de conserver une source de chiffres unique et datée.

3. Architecture et conception

3.1 Backend

L'API est un monolithe modulaire Spring Boot organisé par domaine : authentication, topic, post, comment et user. Chaque domaine regroupe les contrôleurs, DTO, services, dépôts et modèles nécessaires. Les préoccupations transverses, telles que la sécurité, les erreurs, les validations et la configuration, sont isolées dans system.

Cette organisation rend lisible le flux controller, service, repository et évite d'exposer les entités de persistance dans l'API. Flyway versionne le schéma et les données initiales, tandis que Testcontainers isole la base de données des tests d'intégration.

3.2 Performance des accès aux données

Le fil récupère les articles des thèmes suivis en chargeant explicitement leur auteur et leur thème dans la requête `PostRepository.findAllForSubscribedTopics`. Le détail applique le même principe pour l'article, et `CommentRepository.findAllByPostIdWithAuthor` charge les auteurs des commentaires. Cette approche évite les requêtes N+1 lors de la construction des réponses.

La liste des thèmes charge les identifiants des abonnements de l'utilisateur, puis les thèmes, avant de déterminer l'état d'abonnement en mémoire. Ces choix sont adaptés au MVP ; l'absence de pagination sur le fil reste une limite pour un volume de données plus important.

3.3 Frontend

Le frontend Angular est organisé par fonctionnalités : auth, topic, post et user, avec un dossier shared pour les composants réutilisables. Les composants sont standalone, le typage TypeScript est strict, les services HTTP sont typés et la détection de changement est zoneless.

4. Sécurité et protection des données

L'authentification utilise un JWT signé placé dans un cookie HttpOnly. Le frontend ne lit pas le jeton : il rétablit la session avec GET /api/users/me. Les opérations qui modifient des données conservent la protection CSRF au moyen du cookie XSRF-TOKEN et de l'en-tête X-XSRF-TOKEN.

Les mots de passe sont hachés avec BCrypt. Les secrets et mots de passe de configuration sont hors du dépôt, dans un fichier .env local créé à partir de .env.example. Les validations, les contrôles d'accès liés à l'utilisateur courant et la gestion centralisée des erreurs limitent les entrées invalides et la divulgation d'informations techniques.

5. Qualité du code et corrections réalisées

Correction ou amélioration	Décision et impact
Règle de mot de passe centralisée	Validation extraite dans une contrainte backend réutilisable.
Validation frontend simplifiée	Règle de complexité typée et réutilisée par les formulaires.
Filtre JWT lisible	Référence de méthode à la place d'une lambda inutile.
Tests plus lisibles	Assertions et scénarios inatteignables corrigés.
Navigation responsive	Liens partagés desktop-mobile et déconnexion distincte.
Accessibilité du menu mobile	Clic extérieur, Échap et sélection d'un lien vérifiés.
Documentation	README, Javadoc, Swagger UI et Bruno actualisés.

Spotless contrôle le formatage Java ; le frontend dispose d'ESLint et Prettier. Les rapports de couverture et l'audit SonarQube Cloud font partie des outils de suivi de qualité configurés localement.

6. Tests et automatisatisation

- JUnit et Mockito pour les comportements backend ciblés.
- Spring Boot, MockMvc et Testcontainers MySQL pour les intégrations backend.
- Vitest pour les tests unitaires et d'intégration frontend.
- Cypress pour les parcours navigateur.
- Bruno pour le contrat API, les cookies et le CSRF dans une stack isolée.

Les résultats finaux et les taux de couverture sont centralisés dans le rapport de tests et de couverture.

7. Axes d'amélioration et recommandations

7.1 Évolutions fonctionnelles

Axe d'amélioration constaté	Recommandation
Aucun parcours de mot de passe oublié.	Ajouter une réinitialisation par e-mail avec jeton limité et à usage unique.
E-mail non vérifié à l'inscription.	Envoyer un e-mail de validation après l'inscription.
Modification sensible sans mot de passe actuel.	Demander le mot de passe actuel avant un changement sensible.
Fil non paginé.	Ajouter une pagination documentée.
Adresse d'article limitée à son identifiant.	Ajouter un slug à côté de l'identifiant.

7.2 Améliorations techniques et d'exploitation

Axe d'amélioration constaté	Recommandation
Contrôles manuels.	Ajouter une intégration continue.
Aucune mesure de charge.	Établir des mesures avant une hausse du volume.
Chaînes UI dispersées.	Externaliser les libellés et messages pour préparer l'internationalisation.
Cycle de vie des données non formalisé.	Définir conservation, export et suppression avant production.
Pas de supervision d'exploitation.	Préciser alertes, journalisation et gestion des incidents.

8. Conclusion

La revue confirme que le code livré est structuré, documenté et contrôlé en cohérence avec un MVP. Les ajustements réalisés améliorent la sécurité, la cohérence des formulaires, la lisibilité des tests, l'accessibilité de la navigation et l'efficacité des accès aux données. Les évolutions proposées sont distinguées du périmètre livré afin de ne pas les présenter comme des fonctionnalités existantes.